



PEDYA

Delivery, la industria en auge a nivel mundial

Ten tu almuerzo listo con nuestra
aplicación web de delivery de comidas





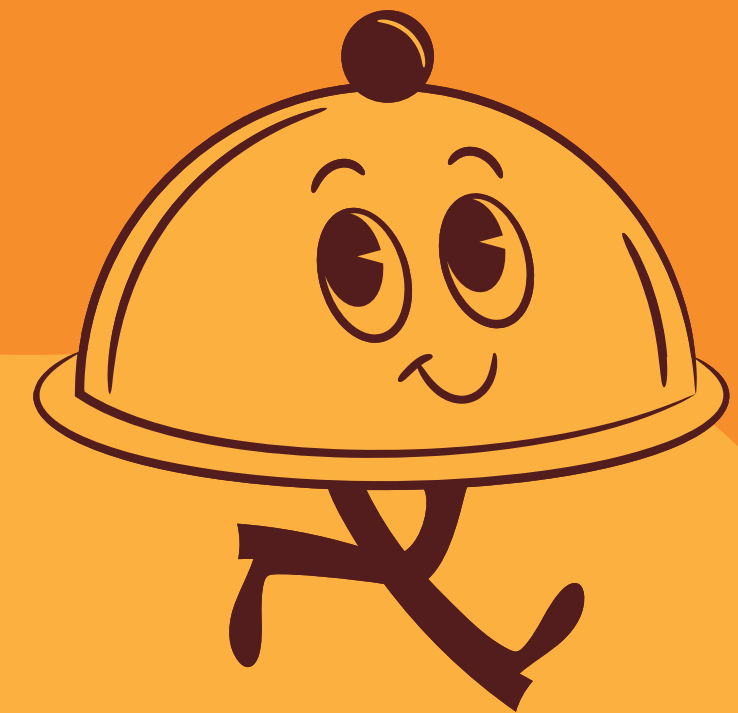
INTEGRANTES

- Fasani Federico
- Gabetta Martin
- Martini Leopoldo
- Scienza Gaspar
- Zegatti Tomas

PedYa

Objetivo

Ofrecer una alternativa moderna al negocio de venta de comida rápida a domicilio basada en tecnologías 100% libres y de código abierto.



¿Que es PedYa?



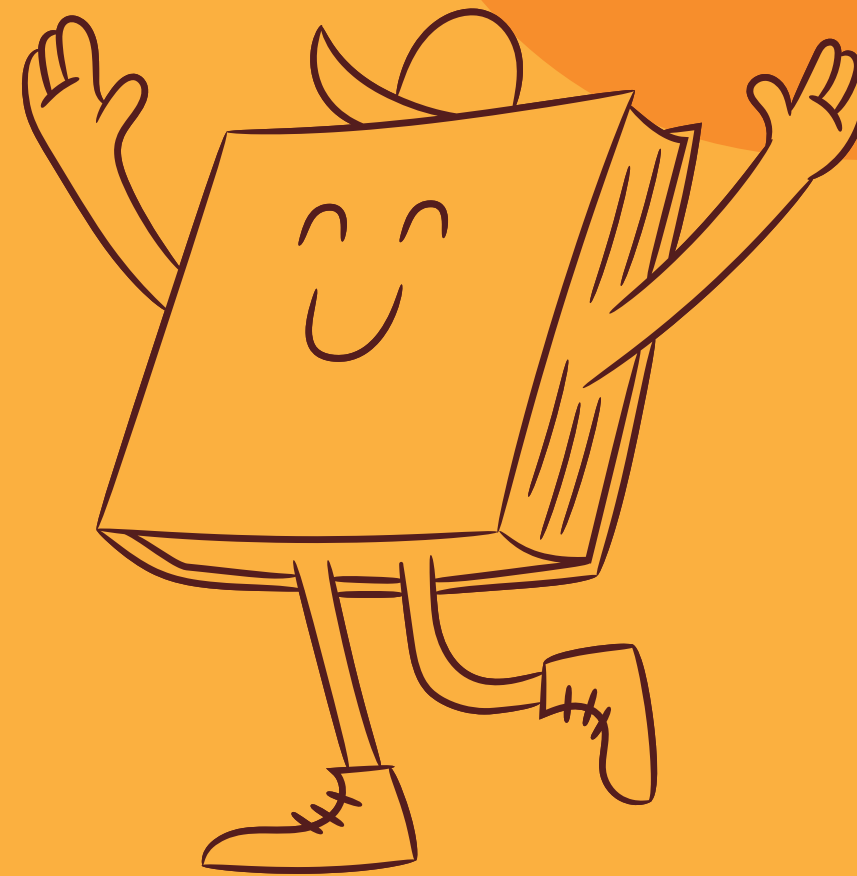
PEDYA

PedYa es una aplicación web de realización y gestión de pedidos de comida rápida que permite a Compradores realizar pedidos a Empresas y que estos sean tomados por Repartidores para realizar su envío.

Alcance

Dentro del alcance

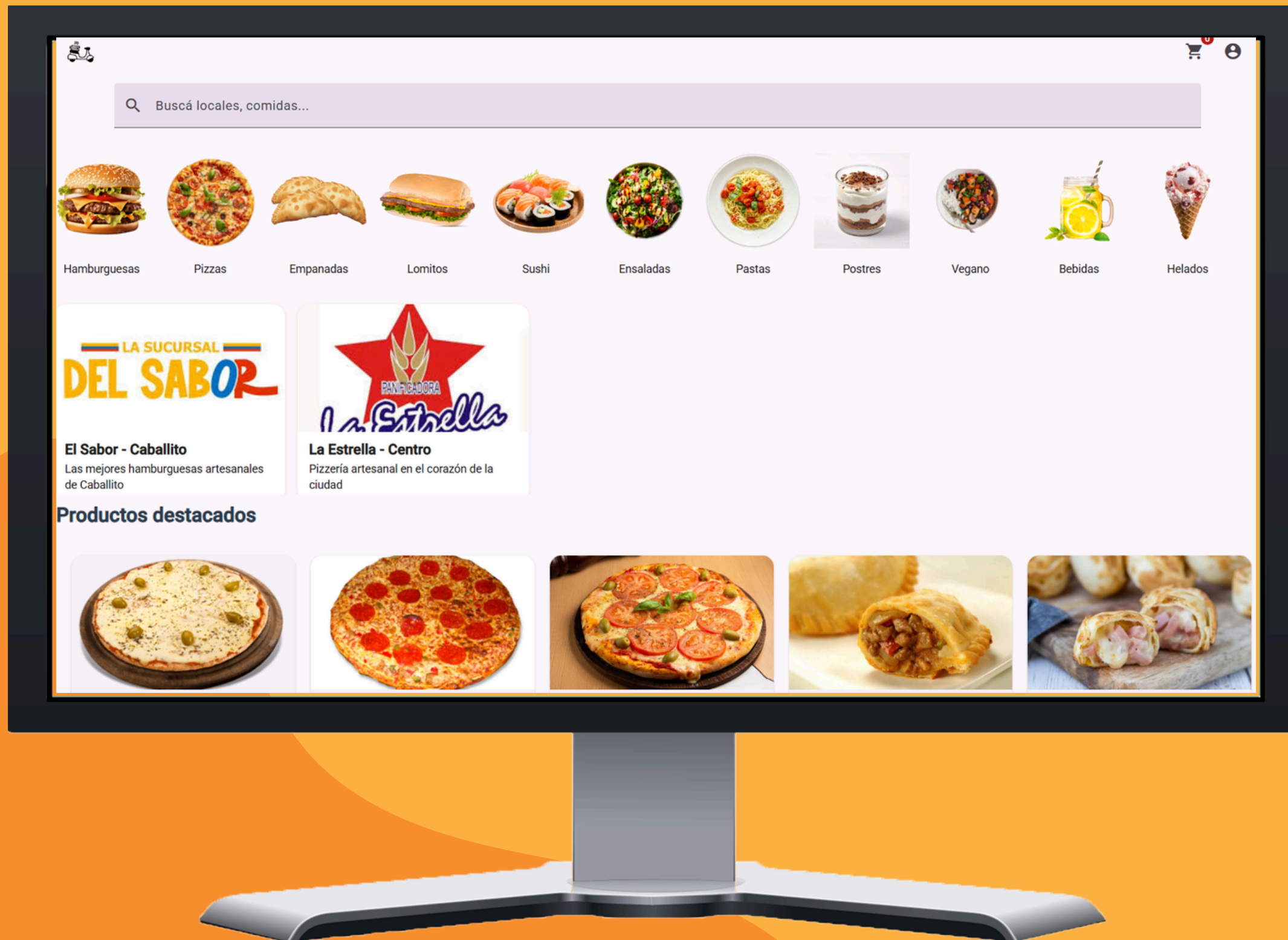
- Creación de diferentes usuarios
- Administración de carrito
- Gestión de productos
- Gestión de Pedidos
- Reportes y estadísticas



Fuera del alcance

- Facturación y cobro
- Gestión de múltiples sucursales por cuenta Empresa
- Interfaz de carga de sucursal

Pantalla Home



Interfaz principal de la aplicación al inicio de la sesión

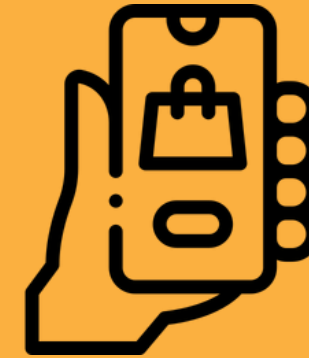


FUNCIONALIDADES/MODULOS



Reparto

Repartidores toman pedidos publicados para realizar su envío.



Publicación de pedidos

Los pedidos que terminan su preparación son publicados por una Empresa para ser tomados por un Repartidor.



Empresas

Cuentas empresa ofertan sus productos a través de la aplicación



Armado de carrito

Compradores pueden armar su carrito de forma rápida, simple e intuitiva.



Pedidos

Seguimiento completo del estado del pedido en tiempo real.



Entregas eficientes

Mediante algoritmos de búsqueda, se obtiene la ruta optima para realizar la entrega de cada pedido.

USUARIOS PRINCIPALES



REPARTIDORES

Usuario con la capacidad de tomar pedidos publicados por empresas para su envío asistido por un mapa de ruta.



EMPRESAS

Usuario con acceso a un ABM de productos, recibe Pedidos y gestiona su preparación y su publicación para que sean tomados por un Repartidor.



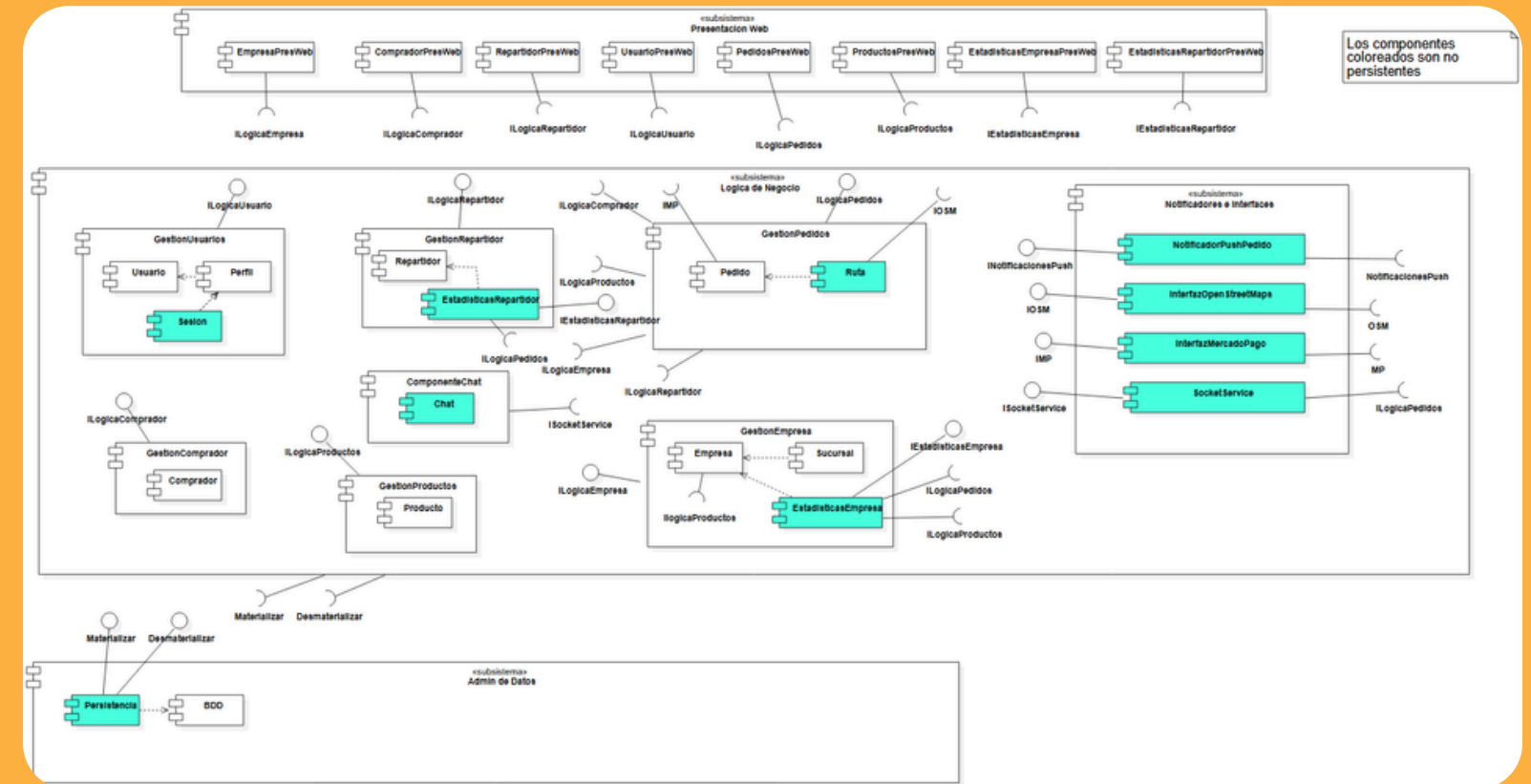
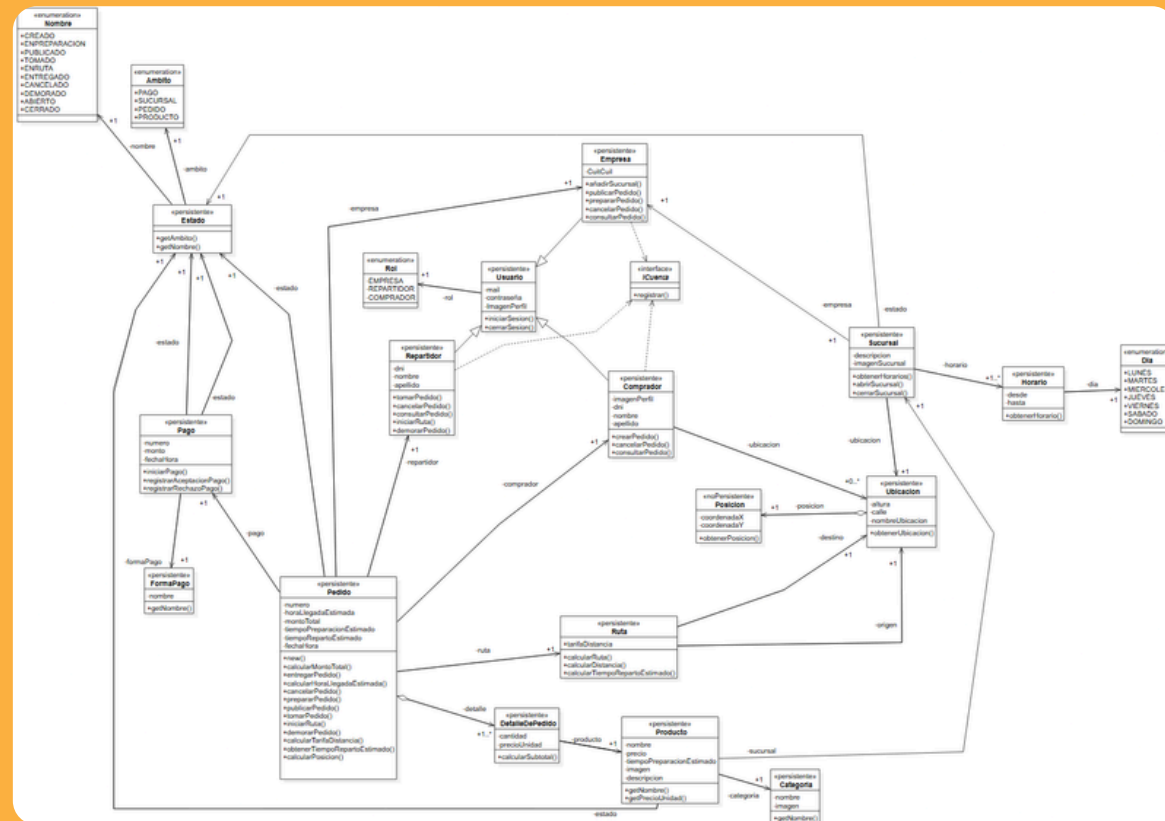
COMPRADORES

Usuario con la capacidad de acceder a la oferta de productos de diferentes Empresas y generar Pedidos a partir de un carrito.

REQUERIMIENTOS NO FUNCIONALES SIGNIFICATIVOS PARA LA ARQUITECTURA

Nro	Nombre
1	Despliegue Web
3	Conexion con Open Street Map
4	Autenticacion de usuarios
9	Seguimiento en tiempo real
14	Base de datos MySQL





DISEÑO Y ARQUITECTURA

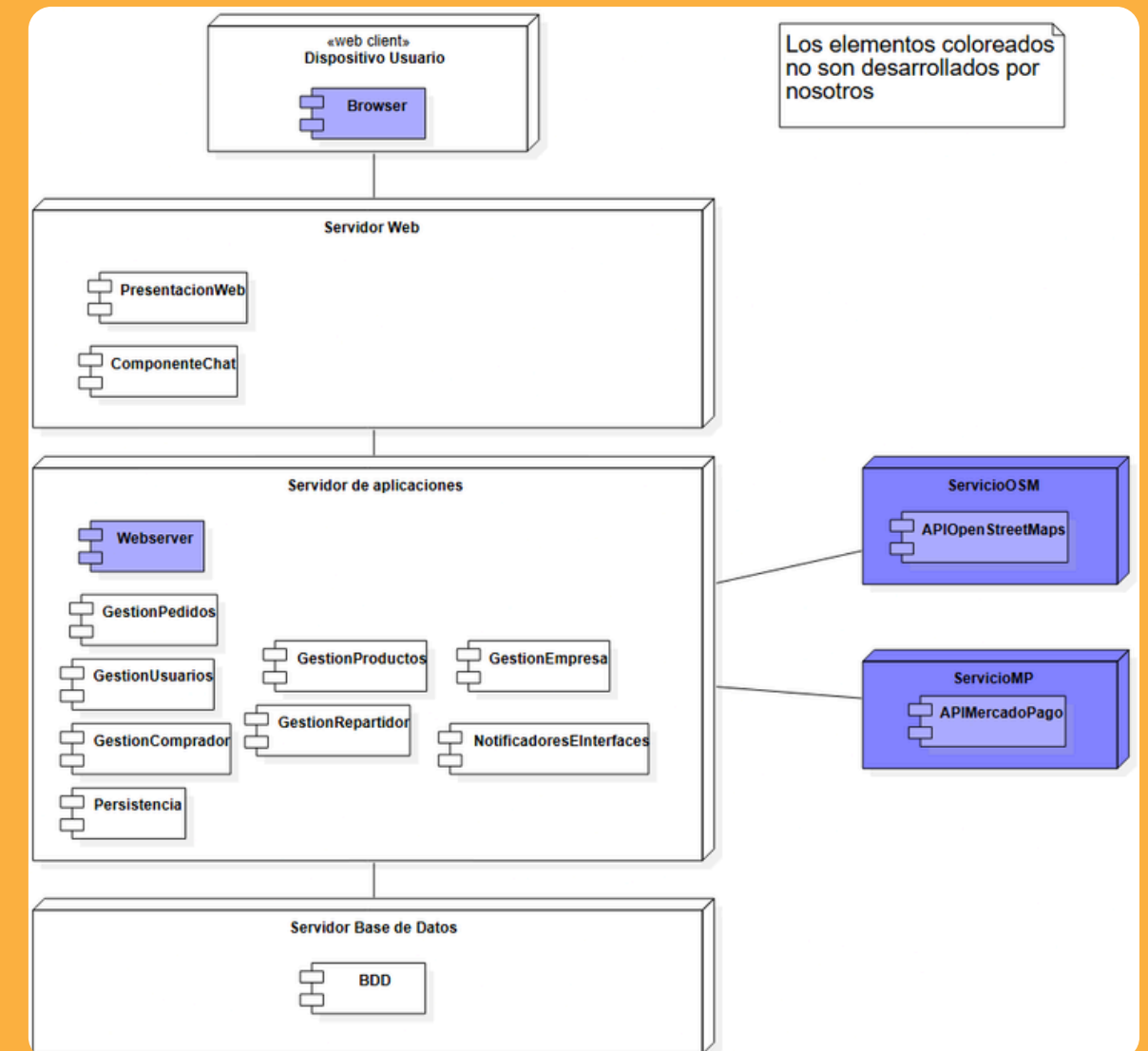
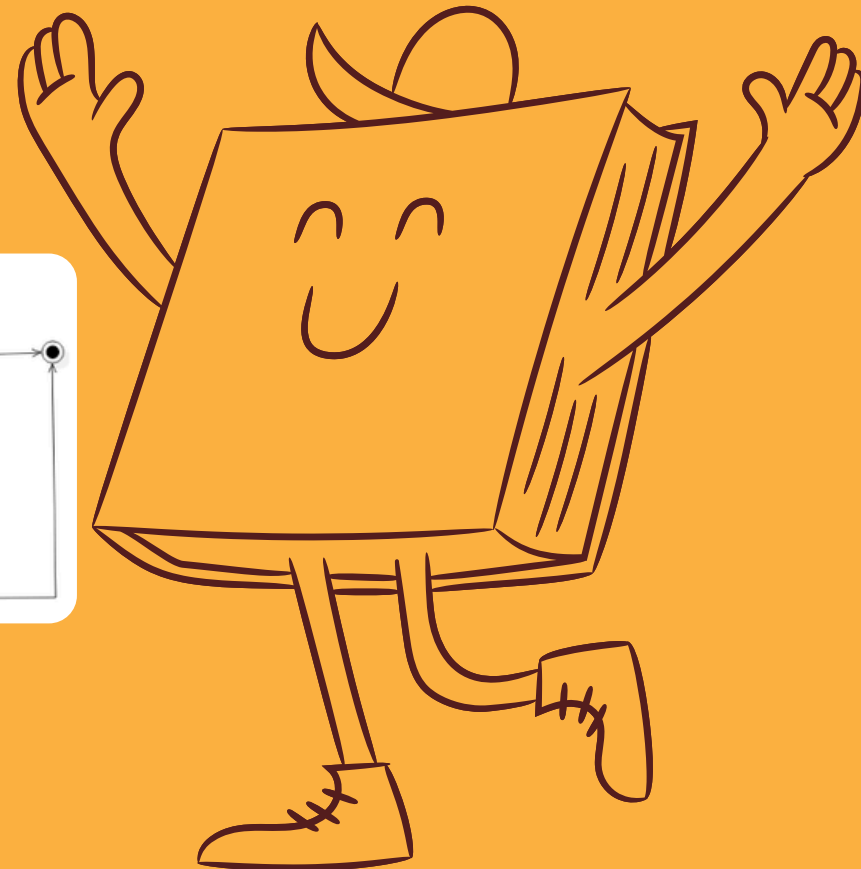
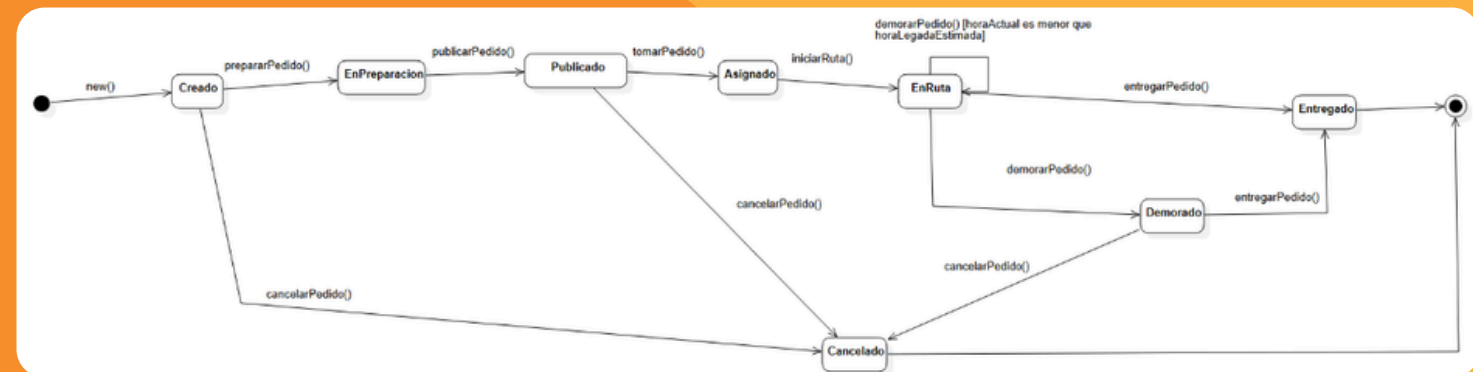


Diagrama de clases

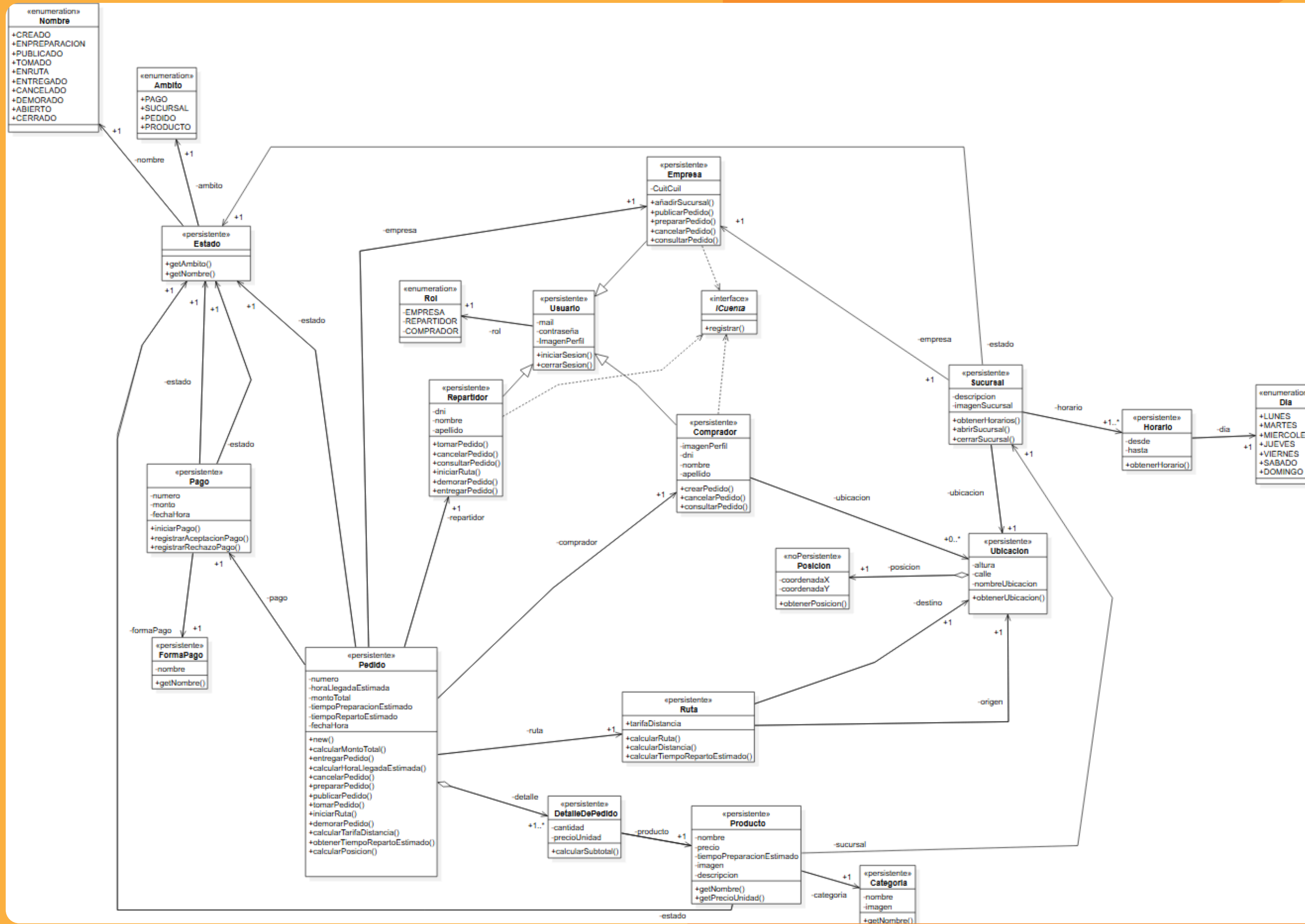


Diagrama de Casos de Uso

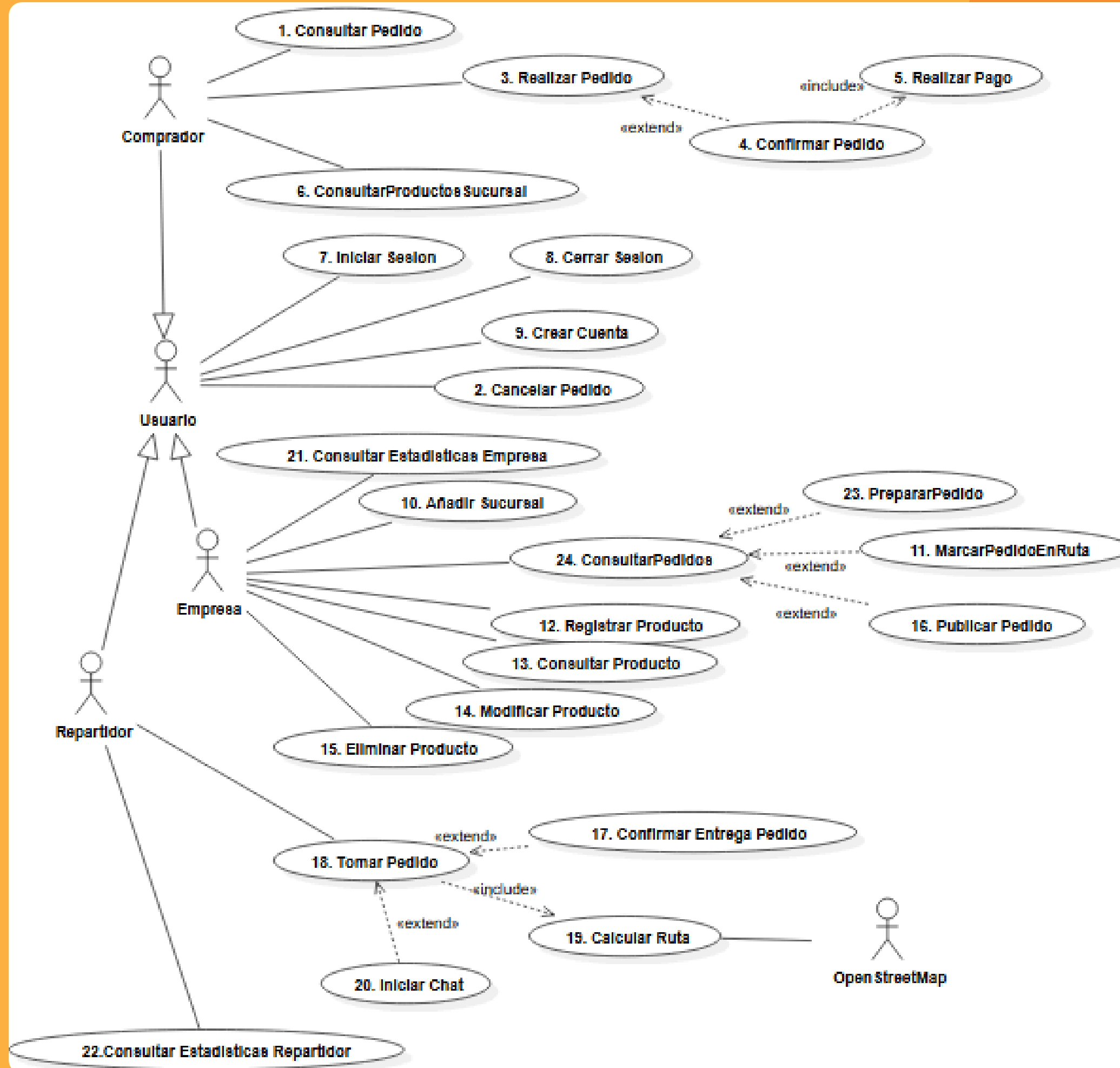


Diagrama de Estados

Clase Pedido



MAQUINA DE ESTADOS CLASE PEDIDO

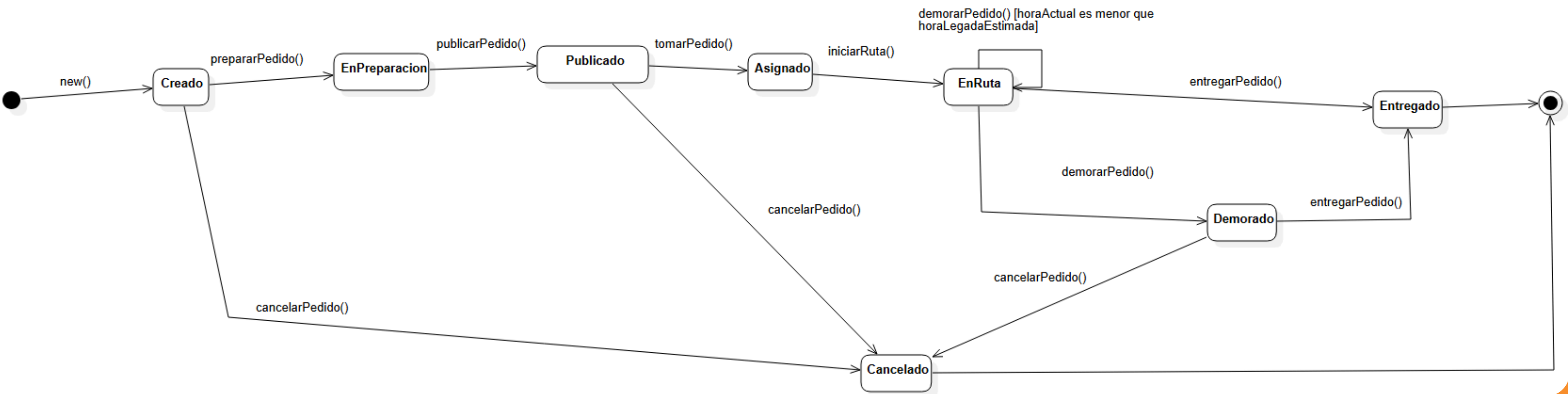


Diagrama de Componentes

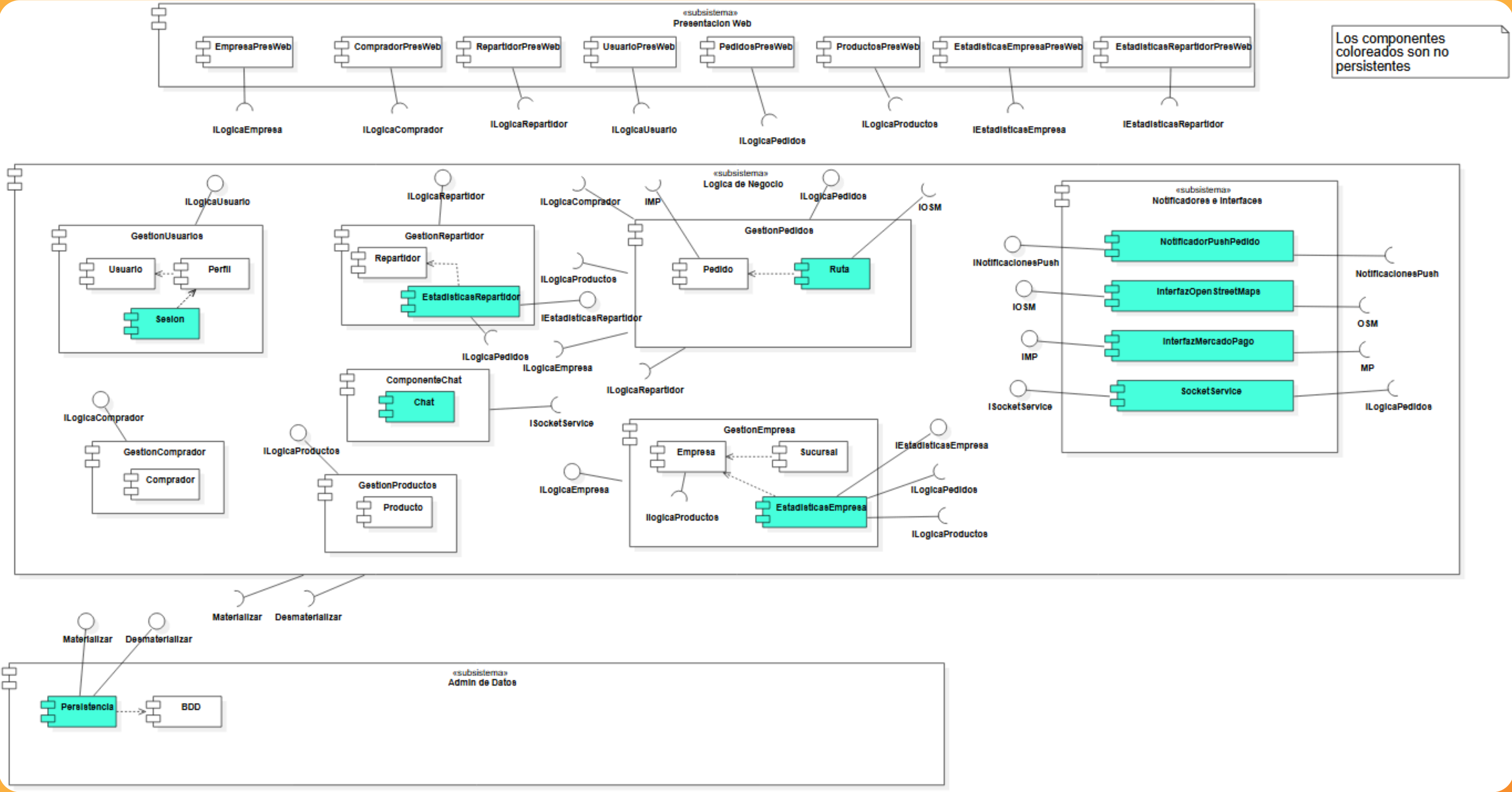
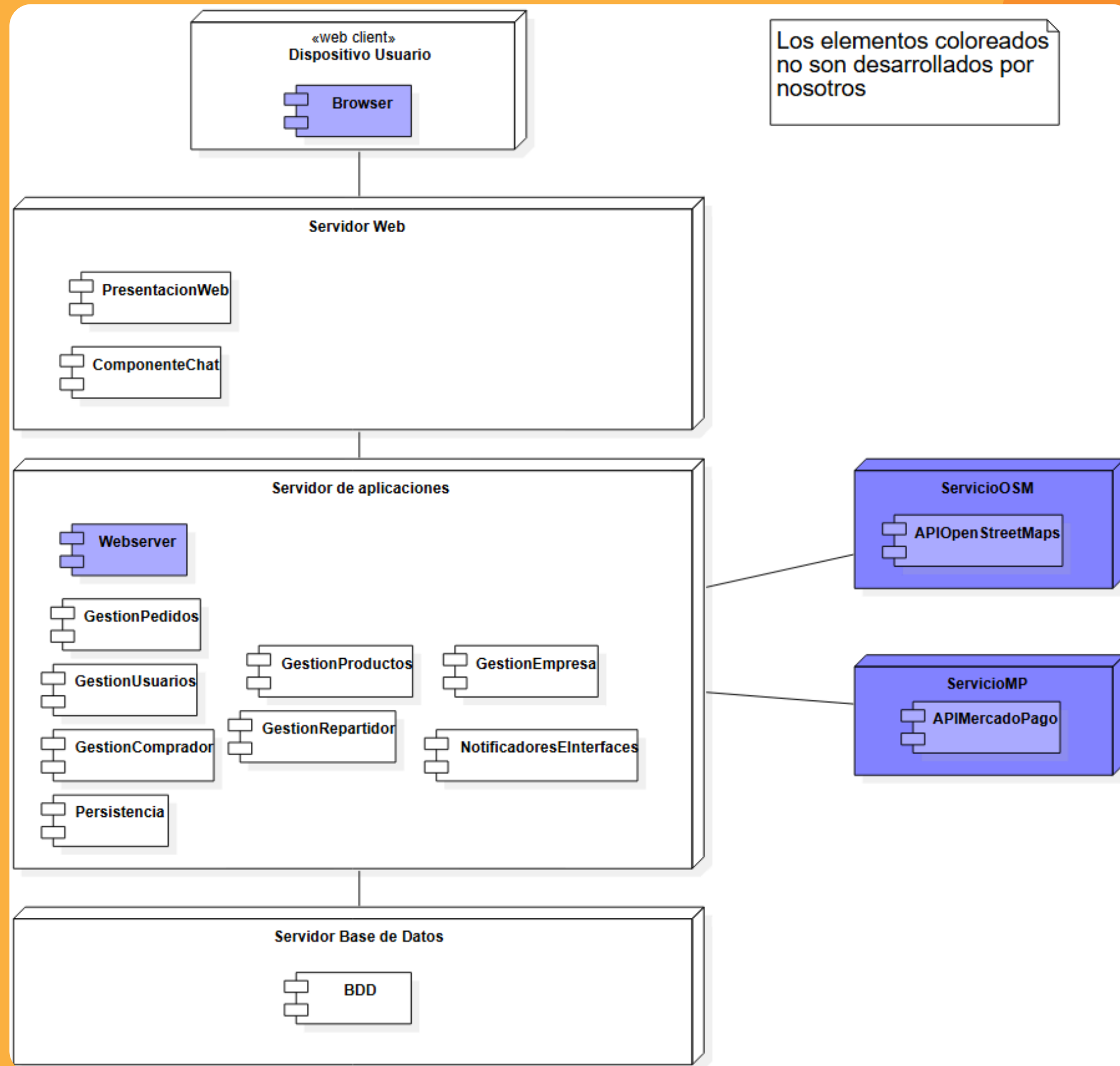


Diagrama de Despliegue



TECNOLOGIAS UTILIZADAS PARA LA APLICACION

BACKEND



Documentacion de endpoints y payloads, donde nos permite realizar solicitudes para verificar respuestas del servidor



ORM utilizado en nuestra aplicacion para la administracion y gestion de los datos los cuales se encuentran en nuestra BD



API incluida en nuestra app para el manejo de mapas interactivos para manejar la ubicación del cliente y el pedido, que el repartidor deberá consultar



Framework utilizado para desarrollo de codigo backend utilizando TypeScript como lenguaje



Base de Datos utilizada en nuestra aplicacion a la cual consultaremos y gestionaremos

CONTROL DE VERSIONES Y TAREAS



TECNOLOGIAS UTILIZADAS PARA LA APLICACION

FRONTEND



Angular

Framework utilizado para el desarrollo del frontend de la aplicación, permitiendo construir una interfaz web dinámica, modular y escalable mediante el uso de componentes reutilizables.

Angular Material

Biblioteca de componentes visuales basada en Material Design, utilizada para crear una interfaz moderna, intuitiva y responsive, mejorando la experiencia del usuario.



Biblioteca utilizada para la comunicación en tiempo real entre el frontend y el backend, permitiendo actualizar automáticamente el estado de los pedidos sin necesidad de recargar la página.



Tecnología utilizada para la integración de mapas interactivos, permitiendo visualizar ubicaciones, calcular rutas y realizar el seguimiento de los pedidos durante la entrega.

DESARROLLO BACKEND ESTRUCTURA GENERAL

```
src
├── auth
├── categoria
├── chat
├── comprador
├── detalle-pedido
├── empresa
├── file
├── forma-pago
├── horario
├── pago
├── pedido
└── posicion
```

```
pedido
├── dto
│   ├── create-pedido.dto.ts
│   └── update-pedido.dto.ts
├── entities
│   ├── pedido.entity.ts
│   ├── pedido.controller.spec.ts
│   ├── pedido.controller.ts
│   ├── pedido.module.ts
│   ├── pedido.service.spec.ts
│   └── pedido.service.ts
```

```
websocket
├── pedido.gateway.ts
├── app.module.ts
└── main.ts
```

```
import { StrategyModule } from '../strategy/strategy.module';
import { ChatModule } from '../chat/chat.module';

@Module({
  imports: [
    PrismaModule,
    UsersModule,
    AuthModule,
    ConfigModule.forRoot({
      isGlobal: true,
    }),
    SucursalModule,
    HorarioModule,
    PagoModule,
    FormaPagoModule,
    PedidoModule,
    DetallePedidoModule,
    CategoriaModule,
    ProductoModule,
    CompradorModule,
    EmpresaModule,
    UbicacionModule,
    PosicionModule,
    RepartidorModule,
    StrategyModule,
    ChatModule,
  ],
  controllers: [FileController],
  providers: [],
})
export class AppModule {}
```

ESTRUCTURA DE LOS MODULOS

SERVICE

Donde se encuentra la lógica de negocio y las reglas de la aplicación realizando una comunicación con la BD

```
@Injectable()
export class PagoService {
  constructor(private readonly prisma: PrismaService) {}

  async create(createPagoDto: CreatePagoDto) {
    const estadoPendiente = await this.prisma.estado.findFirst({
      where: { ambito: 'PAGO', nombre: 'PENDIENTE' },
    });

    if (!estadoPendiente) {
      throw new HttpException(
        'Estado PAGO/PENDIENTE no encontrado en la base de datos',
        HttpStatus.INTERNAL_SERVER_ERROR,
      );
    }

    return this.prisma.pago.create({
      data: {
        numero: createPagoDto.numero,
        monto: createPagoDto.monto,
        fechaHora: createPagoDto.fechaHora ?? new Date(),
        estadoId: estadoPendiente.id,
      },
    });
  }
}
```

CONTROLADOR

Es el encargado de recibir las peticiones HTTP (GET, POST, PUT, DELETE, etc.) que llegan desde el cliente

```
@Controller('pago')
export class PagoController {
  constructor(private readonly pagoService: PagoService) {}

  @Post()
  @Public()
  create(@Body() createPagoDto: CreatePagoDto) {
    return this.pagoService.create(createPagoDto);
  }

  @Get()
  @ApiResponse({ type: Pago, isArray: true })
  findAll() {
    return this.pagoService.findAll();
  }

  @Get('/:id')
  @ApiResponse({ type: Pago })
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.pagoService.findOne(id);
  }
}
```

ESTRUCTURA DE LOS MODULOS

DTO (OBJETOS DE TRANSFERENCIA DE DATOS)

Donde se define cuales, y de que tipo, serán los atributos obligatorios para la creación de una categoría en este caso

```
export class CreateCategoriaDto {  
  @IsString()  
  @IsNotEmpty()  
  @ApiProperty({ required: true })  
  nombre: string;  
  
  @IsDate()  
  @IsNotEmpty()  
  @ApiProperty({ required: true })  
  fechaHora: Date;  
  
  @IsNotEmpty()  
  @ApiProperty({ required: true, type: () => [Number] })  
  productoIds: number[];  
}
```

```
@Module({  
  imports: [PrismaModule, ProductoModule],  
  controllers: [CategoriaController],  
  providers: [CategoriaService],  
})  
export class CategoriaModule {}
```

MODULO

Organización completa de los
archivos utilizados

ENTIDAD

Donde se mostraran todos los
atributos que tendrá la clase
perteneciente a este modulo junto
a su tipo de dato

```
export class Pedido {  
  id: number;  
  numero: string;  
  horaLlegadaEstimada: Date;  
  montoTotal: number;  
  tiempoPreparacionEstimado: number;  
  tiempoRepartoEstimado: number;  
  fechaHora: Date;  
  compradorId: number;  
  repartidorId: number | null;  
  empresaId: number;  
  rutaId: number;  
  pagoId: number;  
  estadoId: number;  
}
```

ESTRUCTURA DE LOS ROLES DEL CLIENTE

PATRON STRATEGY

```
▼ strategy
  ▼ implementations
    TS comprador.strategy.ts
    TS empresa.strategy.ts
    TS repartidor.strategy.ts
  ▼ interfaces
    TS IRegister.interface.ts
  TS strategy.factory.ts
  TS strategy.module.ts
```

```
@Injectable()
export class StrategyFactory {
  constructor(
    private compradorStr: CompradorStrategy,
    private empresaStr: EmpresaStrategy,
    private repartidorStr: RepartidorStrategy,
  ) {}

  getStrategy(role: string): IRegister {
    switch (role) {
      case 'COMPRADOR':
        return this.compradorStr;
      case 'EMPRESA':
        return this.empresaStr;
      case 'REPARTIDOR':
        return this.repartidorStr;
      default:
        throw new Error(`Rol no soportado: ${role}`);
    }
  }
}
```

```
export interface IRegister {
  registerUser(registerDto: CreateUserDto): Promise<any>;
  buscarNombre(id: number): Promise<any>;
  getPersonById(id: number): Promise<any>;
}
```

```
@Injectable()
export default class CompradorStrategy implements IRegister {
  constructor(
    private prisma: PrismaService,
    private userService: UsersService,
    private posicionSvc: PosicionService,
    private ubicacionSvc: UbicacionService,
    private compradorSvc: CompradorService,
  ) {}

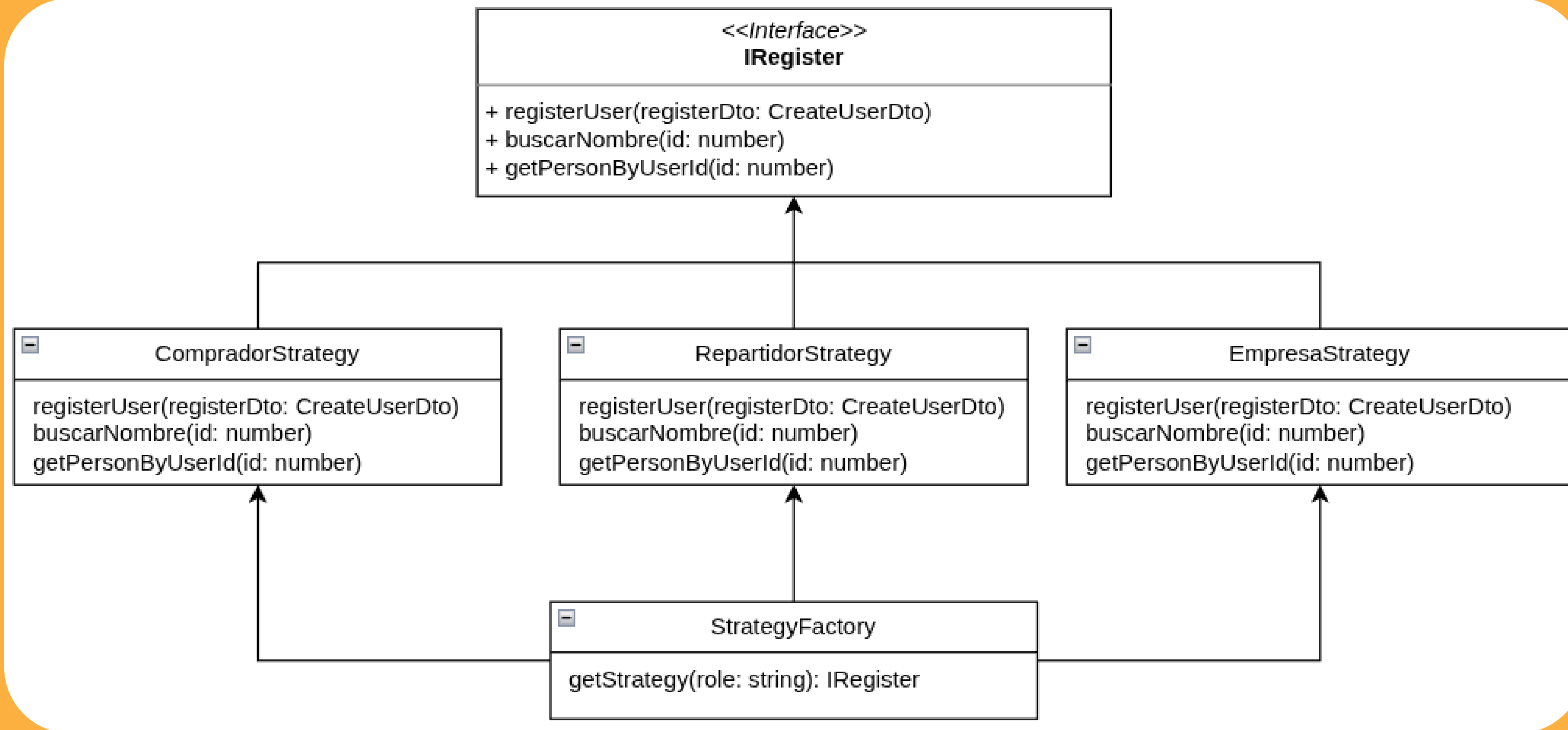
  async registerUser(registerDto: CreateUserDto): Promise<any> { ...
  }

  async buscarNombre(id: number): Promise<any> {
    return await this.compradorSvc.getCompradorById(id);
  }

  async getPersonById(id: number): Promise<any> {
    return await this.compradorSvc.getCompradorIdById(id);
  }
}
```

DIAGRAMA

PATRON STRATEGY



USO DEL PATRON

PATRON STRATEGY

```
@Injectable()
export class AuthService {
  public comprador!: Comprador;
  public empresa!: Empresa;
  public estrategia!: IRegister;

  constructor(
    private readonly jwtService: JwtService,
    private readonly prisma: PrismaService,
    private readonly userService: UsersService,
    private strFactory: StrategyFactory,
  ) {}

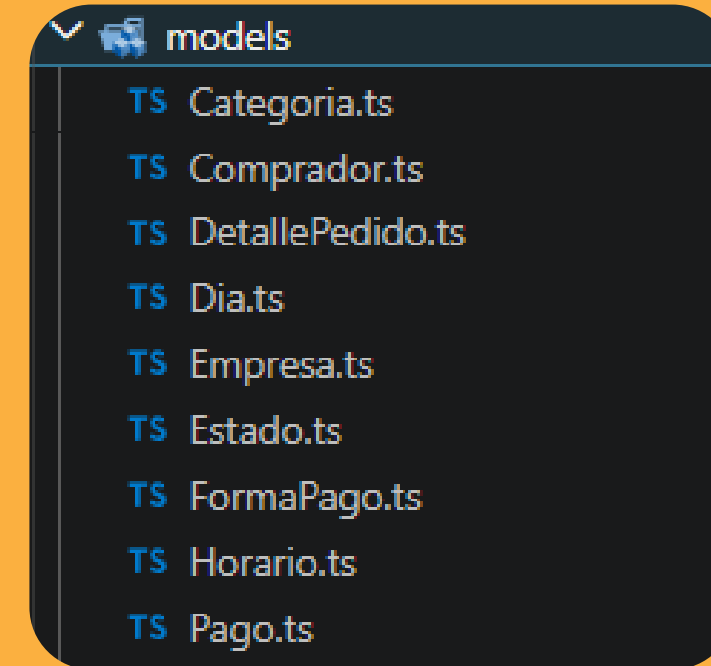
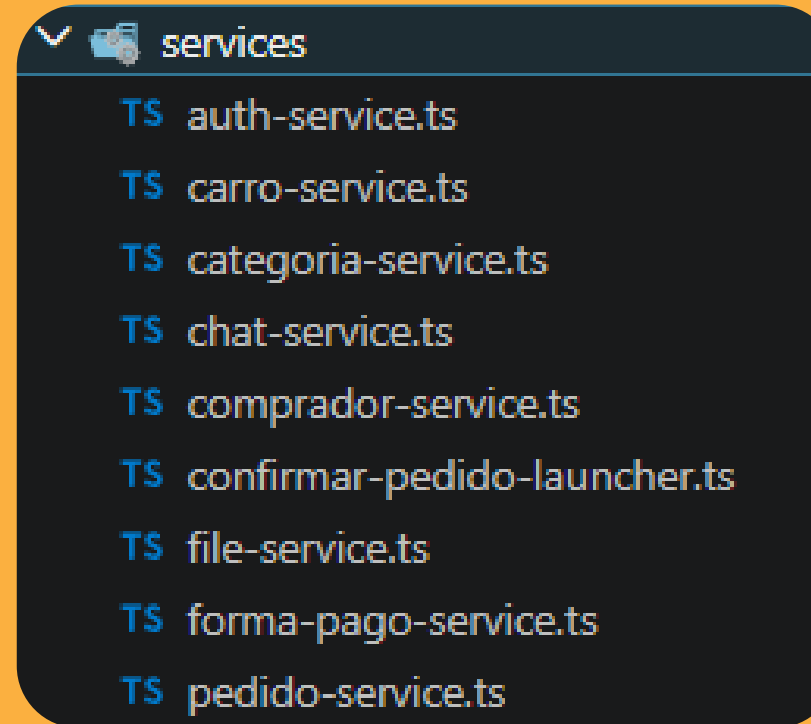
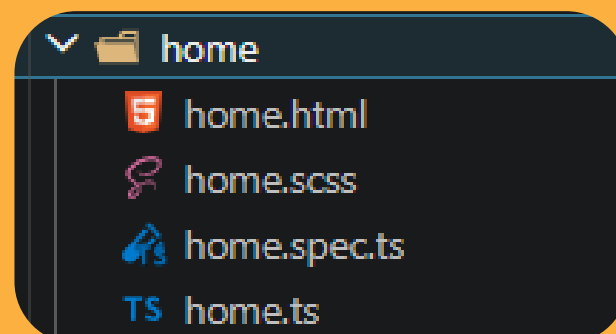
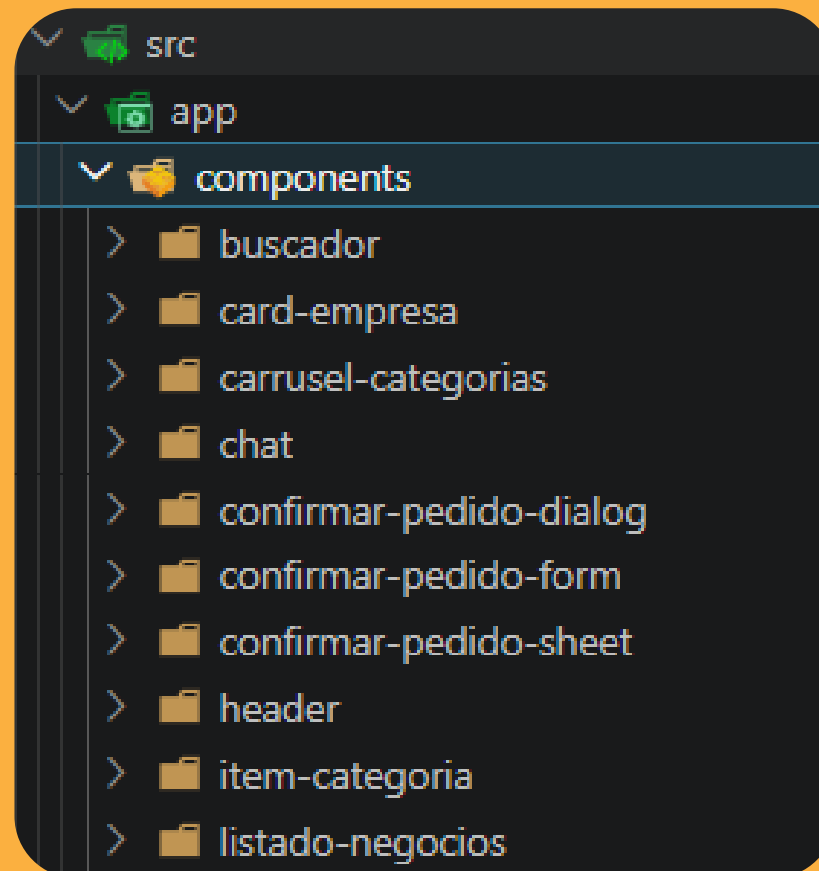
  // Validar usuario y generar token JWT
  async validateUser(loginDto: LoginDto) { ...
  }

  async registerUser(registerDto: CreateUserDto) {
    const { mail, contrasena, rol } = registerDto;
    this.estrategia = this.strFactory.getStrategy(rol!.toString());
    await this.estrategia.registerUser(registerDto);
    const postValuesValidate = {
      mail,
      contrasena,
    };
    const userToken = await this.validateUser(postValuesValidate);
    return userToken;
  }

  async getProfile(req: Request) { ...
  }
}
```

DESARROLLO FRONTEND

ESTRUCTURA GENERAL



```
export const routes: Routes = [
  {path: 'home' ,component: Home},
  {path: 'login' ,component: Login},
  {path: 'categoria/:id' ,component: ItemCategoria},
  {path: 'registro' ,component: Registro},
  {path: 'sucursal/:idEmpresa' ,component: SucursalConsultaComponent},
  {path: 'seguimiento/:pedidoId' ,component: SeguimientoPedido},
  {path: 'seguimiento' ,component: SeguimientoPedido},
  {path: 'seleccion-ubicacion' , component: SeleccionUbicaciones},
  {path: 'tabla-pedidos' , component: TablaPedidos},
  {path: 'categoria/:idCategoria/productos' , component: ProductosPorCategoria},

  {path: 'producto' ,component: ProductoTableComponent},
  {path: 'chat' ,component: Chat},
  {path: 'repartidor' , component: RepartidorComponent},

  {path: '**' ,redirectTo: 'home'}
];
```

ESTRUCTURA DE LOS MODULOS

```
@Component({ ...
})
export class Home implements OnInit {
  private authService = inject(AuthService);
  private productoService = inject(ProductoService);
  private carroService = inject(CarroService);
  private pedidoLauncher = inject(ConfirmarPedidoLauncher);
  private router = inject(Router);

  productos = signal<Producto[]>([]);
  totalItemsCarrito = computed(() => this.carroService.getTotalItems());

  ngOnInit() {
    this.productoService.getProductos().subscribe({
      next: (data) => this.productos.set(data),
      error: console.error,
    });
  }

  abrirCarrito() {
    const sucursal = this.carroService.getItems()[0]?.producto.sucursal;
    if (!sucursal) return;
    this.pedidoLauncher.abrir(sucursal);
  }

  get isEmpresa(): boolean {
    return this.authService.isEmpresa();
  }

  get isRepartidor(): boolean {
    return this.authService.hasRole(Rol.REPARTIDOR);
  }
}
```

COMPONENTE

Donde se realizara la lógica y solicitara los métodos al service para manejar la funcionalidad de la pantalla

```
@Injectable({
  providedIn: 'root'
})
export class PedidoService {
  private socket: Socket;
  private pedidosSubject = new BehaviorSubject<Pedido[]>([]);
  public pedidos$ = this.pedidosSubject.asObservable();

  private apiUrl = `${environment.apiUrl}/pedidos`;

  constructor(...
  )

  crearPedido(pedido: any): Observable<Pedido> {
    return this.http.post<Pedido>(`${this.apiUrl}`, pedido);
  }

  crearPago(data: { numero: string; monto: number; formaPagoId?: number }): Observable<{ id: number }> {
    return this.http.post<{ id: number }>(`${environment.apiUrl}/pago`, {
      ...data,
      fechaHora: new Date().toISOString(),
    });
  }

  findOne(pedidoId: number): Observable<Pedido> {
    return this.http.get<Pedido>(`${this.apiUrl}/${pedidoId}`);
  }
}
```

SERVICE

Encargado de comunicar el backend con el frontend, definiendo los datos que se tomaran como entrada y los que ofrecerá de salida para utilizarlo en los componentes

```
<app-header></app-header>

<!-- Vista EMPRESA -->
@if (isEmpresa) {
  <div class="empresa-home">
    <div class="empresa-header">
      <h2>Bienvenido, {{ nombreUsuario }}</h2>
      <p class="subtitulo">¿Qué querés gestionar hoy?</p>
    </div>
    <div class="dashboard-grid">
      <mat-card class="dashboard-card" routerLink="/tabla-pedidos">
        <mat-card-content>
          <mat-icon class="card-icon pedidos-icon">receipt_long</mat-icon>
          <h3>Gestión de Pedidos</h3>
          <p>Ver y actualizar el estado de los pedidos en tiempo real</p>
        </mat-card-content>
        <mat-card-actions>
          <button mat-flat-button color="primary">Ir a Pedidos</button>
        </mat-card-actions>
      </mat-card>

      <mat-card class="dashboard-card" routerLink="/producto">
        <mat-card-content>
          <mat-icon class="card-icon productos-icon">storefront</mat-icon>
          <h3>Mis Productos</h3>
          <p>Administrar el catálogo de productos de tu sucursal</p>
        </mat-card-content>
        <mat-card-actions>
          <button mat-flat-button color="accent">Ir a Productos</button>
        </mat-card-actions>
      </mat-card>
    </div>
  </div>
}
```

ARCHIVO HTML

Lo que sera el cuerpo de la parte visual de la pantalla, en donde se harán las llamadas a los métodos definidos en el componente

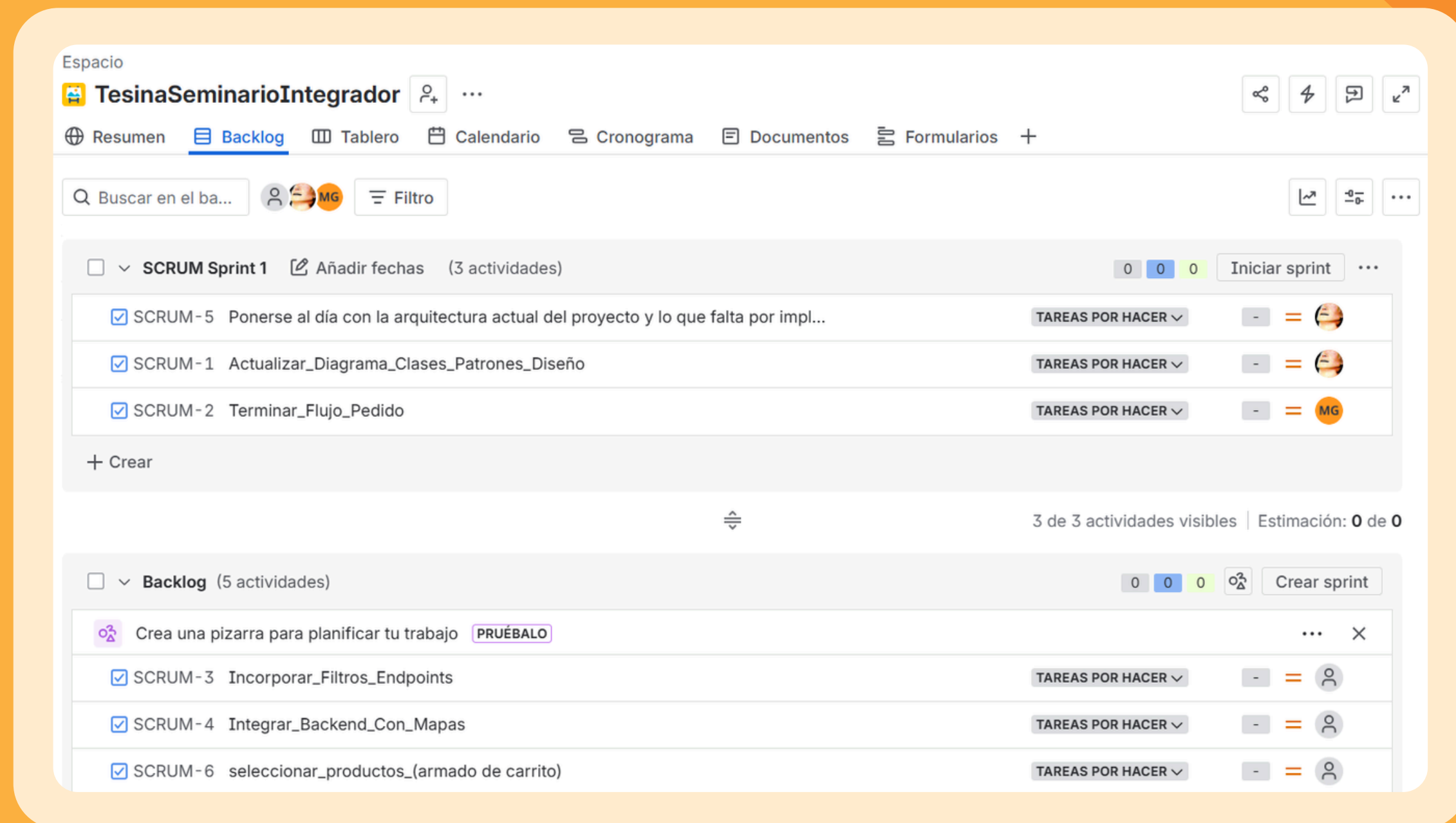
METODOLOGIA DE TRABAJO AGIL

Ciclo de vida iterativo e incremental

Flujo de trabajo basado en scrum

Comunicacion:

- Jira
- Whatsapp
- Discord



PROXIMOS PASOS

Luego de la version MVP del producto, las proximas funcionalidades a implementar seran:

Facturacion

Mediante portales como Mercadopago

Guardado de ubicaciones

Por parte de Clientes

Registro de franquicias

Por parte de Empresas

Notificaciones

Sobre el estado del Pedido





Gracias

**Ahora veamos
la App!**